Critical Thinking Module 5: Virtual Memory Settings on macOS

Alexander Ricciardi

Colorado State University Global

CSC507: Foundations of OSs

Professor: Dr. Joseph Issa

Jun 14, 2026

Critical Thinking Module 5: Virtual Memory Settings on macOS

Operating Systems (OSs) use memory management to coordinate how programs share limited Random Access Memory (RAM) while still keeping the computer responsive, stable, and secure. One of the most important techniques used for this purpose is virtual memory. Virtual memory combines RAM with disk storage so that the system can behave as if more memory is available than is physically installed (Colorado State University Global, n.d). Gupta (2024) makes the same broader point by describing virtual memory as one of the central memory-management techniques used by modern operating systems to improve utilization, protection, and performance. In other words, virtual memory is not a small optional feature. It is one of the main ways that an OS continues functioning smoothly when many applications compete for memory at the same time.

This paper asks whether the current virtual memory setting is optimal, what happens if it is doubled, what happens if it is turned off, how RAM size changes the recommendation, and which applications are affected most. This paper examines those questions on the current host machine rather than on a hypothetical generic computer. The computer used for this evaluation is a Mac Studio running modern macOS. That fact matters because the operating system design changes how the project must be answered. Current macOS allows the user to observe memory pressure, compressed memory, and swap use, but it does not provide a simple user-facing `pagefile box` like the setting commonly discussed on Windows systems (Apple, 2003; Apple, n.d.-a; Apple, n.d.-b; Apple, n.d.-c).

The method used for this paper was intentionally safe and non-destructive. The machine was inspected with `sw_vers, system_profiler SPHardwareDataType SPSoftwareDataType`, `sysctl vm.swapusage`, `vm_stat, memory_pressure`, and process `snapshots`. These commands

made it possible to identify the operating system, hardware, RAM size, secure virtual memory status, swap allocation, and the current workload conditions. This approach fits the practical reality of the platform. It would not be responsible to claim that unsupported memory-management changes were actually performed on a current Mac merely to satisfy a prompt. The logical conclusion that can be made is that this project must be answered through careful observation of the present state and reasoned analysis of what the alternative states would likely do.

Virtual Memory on the Current Mac

Table 1

System identification for the machine

Item	Observed value
Hardware	Mac Studio
Model identifier	Mac15,14
OS	macOS 26.5 (25F71)
Processor	Apple M3 Ultra
System type / architecture	arm64
Installed RAM	96 GB
Secure virtual memory	Enabled

Note: The table shows the machine and OS specifications.

These hardware facts matter because they shape how strongly the computer depends on virtual memory during normal use. A machine with only a modest amount of RAM would likely reach memory pressure quickly while multitasking. By contrast, a Mac Studio with 96 GB of RAM begins from a much stronger position. This does not mean that virtual memory becomes unnecessary. It means that the role of virtual memory changes from frequent survival mechanism to protective reserve for unusually large workloads. The closest equivalent to “locating the virtual memory settings” on this Mac is locating the system views that reveal how macOS is

managing memory automatically. Apple documents that the standard place to inspect this information is the Memory tab in Activity Monitor. There the user can examine Memory Pressure, Physical Memory, App Memory, Wired Memory, Compressed memory, Cached Files, and Swap Used (Apple, n.d.-b). Apple also explains that the same Activity Monitor memory view is the correct place to determine whether the computer is using memory efficiently or whether more RAM would be helpful (Apple, n.d.-a). System Information supplies the hardware details, installed RAM, and secure virtual memory status. In other words, the current Mac does let the user locate and inspect virtual memory behavior, but it does so through monitoring tools rather than through a manual size-setting dialog box. The lower-level system evidence supports the same conclusion. The local `dynamic_pager(8)` manual identifies `dynamic_pager` as the macOS swap configuration daemon and states that `swapfiles` are managed under `/private/var/vm/swapfile*` by default (Apple, 2003). This means that macOS is expected to manage swap capacity as part of normal operating-system behavior. The user can observe the result, but the OS is responsible for orchestrating it. The observed memory snapshot is shown in **Table 2.**

Observed memory snapshot from the current machine

Metric	Observed value
Physical memory	96 GB
Swap currently allocated	2048.00 MB
Swap currently used	493.12 MB
Swap currently free	1554.88 MB
Swap state	Encrypted
System-wide free memory	About 90%
Page size	16 KB

Note: The table shows a snapshot of the machine memory system.

These measurements are important because they reveal the actual current condition of the machine rather than a generic theory. The system is using virtual memory, but only modestly. About 493 MB of swap was in use at the time of observation, which is small relative to 96 GB of installed RAM. The memory pressure result also showed about 90% free system memory. This means that the machine is not presently struggling to keep its active working set in RAM. In other words, the OS has virtual memory available and active, but it is not relying on it heavily to keep ordinary work moving forward. The active application set also helps explain this result. Visible foreground applications included Finder, Google Chrome, Microsoft Word, Mail, Calendar, TextEdit, Numbers, Terminal, Code, Preview, and Codex. A process snapshot showed that Microsoft Word, Google Chrome and its renderer helpers, and Codex renderer/helper processes were among the larger active memory consumers. This distinction matters because modern browser and Electron-style workloads often spread their memory usage across many helper processes instead of one obvious executable. Even so, the machine still had abundant memory headroom. That means the current workload mix was well within the comfortable operating range of the installed RAM.

Are the Current Settings Optimal on This Mac?

For this Mac, the present virtual memory behavior is effectively optimal for normal coursework, browsing, document editing, and coding work. Apple explicitly states that macOS obtains the best performance by efficiently using and managing all of the computer's memory and that unused memory does not automatically improve performance (Apple, n.d.-a). This statement is important because many users assume that the ideal system is one that never touches swap. Apple's guidance points in a different direction. The better question is not whether swap exists, but whether the system is handling memory efficiently overall. The local measurements

support the Apple guidance. The machine has 96 GB of RAM, secure virtual memory enabled, only modest swap use, and high free-memory headroom. This combination suggests that the operating system is not being forced into aggressive paging under the current workload. A page fault creates overhead because the OS must fetch the needed page into memory before the process can continue, and operating-system policy decisions such as demand paging, replacement policy, and cleaning policy all influence the cost of that overhead (Colorado State University Global, n.d). Gupta (2024) makes the same broader point by describing virtual memory and page-replacement strategy as part of the OS effort to balance utilization, scalability, and performance. On this Mac, those costs are currently being kept in check because the installed RAM is large enough to hold the active workload comfortably.

There is also a security reason to call the current setting optimal. Apple explains that secure virtual memory encrypts the data that is moved between RAM and disk and that this protection is always on in macOS (Apple, n.d.-c). This means that the current configuration is not only a performance decision but also a data-protection decision. A different configuration that attempted to bypass standard swap handling would not simply be a tuning experiment. It would also interfere with a normal operating-system protection mechanism. Apple's memory-pressure explanation strengthens the same conclusion. Apple documents that the Memory Pressure graph is the correct summary signal for whether the system is using RAM efficiently: green indicates efficient use, yellow indicates rising pressure, and red indicates insufficient memory for the current demand (Apple, n.d.-a). The command lines used in this project do not reproduce the color graph directly, but they generate values that examine the same RAM conditions. In other words, this Mac is not showing that its memory system needs to be expanded

or optimized. From it, it can be concluded that the current Apple-managed setting of this machine is optimal as it keeps the machine workload stable and responsive.

What Happens if Virtual Memory Is Doubled or Turned Off?

Currently, macOS does not show a user workflow for manually doubling a fixed virtual memory size or disabling virtual memory completely. For that reason, these two scenarios, that is, if the virtual memory is doubled or turned off, must be treated as evaluations rather than as screenshots of setting changes, as macOS does not allow the virtual memory to be modified or turned off. It is important to understand that if the virtual memory were doubled, the result would not be a faster computer; virtual memory is not RAM. In other words, swap is still disk-backed memory rather than true RAM. Page faults add overhead, and that overhead is due to the fact that disk access is slower than RAM access (Colorado State University Global, n.d). A larger swap can help a system avoid immediate failure when memory pressure is very high, but it does not remove the cost of paging. In other words, doubling available swap would likely increase fallback capacity, not operating speed.

This matters because “more available virtual memory” is often confused with “more performance,” and they are two different things. On a system that is experiencing memory starvation, a larger swap can allow the OS to keep more processes open. However, if the system uses paging heavily and memory swapping, the machine usually becomes less responsive because it is spending more time moving memory contents between RAM and storage. Thus, more swap is mostly a stability cushion, not a substitute for RAM performance. On this specific Mac machine, the doubling swap would probably happen rarely during ordinary use, when using word processing apps or browsing the web, for example, because the machine has 96 GB of RAM and low current swap use. The difference would be more relevant during large workloads

such as large software development workspaces and heavy local AI or Machine Learning experiments, or when using virtual machines to test applications. Even then, the advantage would mostly be that the system could tolerate a larger memory spike before failing. It would not mean that those workloads suddenly become faster.

Apple's virtual memory documentation states that virtual memory is always on in macOS and that data moved between RAM and disk remains encrypted (Apple, n.d.-c); therefore, turning it off on this machine is not advised, as the virtual memory is treated by macOS as being part of its normal functionality rather than as an optional feature. Furthermore, turning off virtual memory is not supported by macOS; this is because the system would lose one of its major protections against sudden memory exhaustion. The only way to turn it off is to hack the OS; nonetheless, the hacked OS would still try to rely on compression and other internal memory-management methods, but once physical RAM became exhausted, the machine would have far fewer options. In other words, applications that request large memory blocks could stall, not have enough allocated memory to function properly, be terminated, or crash. Doubling or turning off virtual memory would mainly affect memory-intensive applications, such as video editing apps. Doubling virtual memory would mainly increase fallback capacity for those workflows, while turning virtual memory off would remove important safety measures and make memory system failures more likely.

How RAM Size Changes the Recommendation

RAM size changes the recommendation because virtual memory acts as a safeguard when memory demand exceeds the RAM capacity. For example, systems with smaller amounts of RAM depend much more heavily on swap, on virtual memory, especially when users try to run browsers, office programs, messaging tools, video calls, PDFs, and development tools at the

same time. Such workflows put a lot of pressure on the system memory, page faults become more frequent, and the OS must work harder to keep the system responsive. Gupta (2024) explains that a memory-management system's efficiency depends on its capacity to manage process memory allocation, prevent process memory interference, and control RAM storage swaps. The fetching policy, when to bring a needed page into main memory, and the replacement policy, deciding which page should be removed from RAM, matter because main memory is limited and page faults are expensive (Colorado State University Global, n.d). In other words, the less physical memory a system, RAM, has, the more often virtual memory is used, the slower the system becomes, negatively affecting the performance of the overall system and, by consequence, the user experience. Higher-RAM systems can reduce these negative effects. When more of the active workflow memory requirements can stay within the RAM, the OS performs fewer swaps, and the overhead cost of using virtual memory drops. Apple's memory-pressure approach supports this view because it emphasizes efficient overall use rather than raw unused-memory counts (Apple, n.d.-a; Apple, n.d.-b). This implies that on a machine with large RAM capacities, the virtual-memory system is still relevant, but it spends more time acting as protection against memory-need spikes than as a constant memory-running mechanism required to maintain the system responsive.

With 96 GB of RAM, the Mac machine, with its current workload, most processes requirement need remain largely within the RAM. The low observed swap use shows that the machine is not being forced into frequent heavy paging during the observed tasks. The logical conclusion is that the best recommendation for this Mac is not to enlarge the swap manually or to try to disable it. The best recommendation is to keep the Apple-managed configuration and let the large RAM capacity do most of the work. The same conclusion would not be as strong on a

much smaller machine with much smaller RAM capacity. An old laptop with significantly less RAM would feel the cost of multitasking, impacting the user experience. A workload that combines many browser tabs, a video meeting, large PDFs, office applications, and an IDE would force this older, smaller system to perform memory compression and swap more often. This means the user would likely notice slowdowns in web browser tab switching, when launching applications, and when searching or indexing large files.

Overall, more RAM does not eliminate the need for virtual memory, but it does reduce how often the system must rely on it. Rupabhinda's (2018), in his survey of memory management techniques for virtual machines, reaches a related conclusion by explaining that memory demand is highly application dependent and that dynamic memory-management techniques always involve tradeoffs and performance penalties when memory pressure rises. Kanellopoulos et al. (2025) agree by explaining that virtual-memory overhead remains an important systems bottleneck in demanding workloads, especially when address translation and software overhead grow large over time. These studies support the argument that the question is not only "how much memory is installed?" but also "what kind of workload is being run?" The combination of RAM size and workload shape determines how strongly virtual memory affects performance.

Performance Effects on Different Application Types

Different application types are affected differently by virtual memory because they consume memory in different ways. The observations from this Mac showed that Google Chrome and its many renderer helpers, Codex renderer/helper processes, and Microsoft Word were among the larger active memory consumers during the measurement window, while Code was also active as part of the desktop workload. This pattern is not surprising. Browsers,

Electron-based tools, and development environments often use many helper processes, tabs, renderers, extensions, and services. This means their total memory demand can grow steadily even when the screen does not appear especially busy.

Applications such as Google Chrome, Code, and Codex are therefore among the most sensitive to memory pressure. If virtual memory use rises significantly, these applications are likely to show slower tab switching, renderer pauses, slower project search, slower indexing, and reduced responsiveness. This is especially true when many tabs, windows, workspaces, or background helpers are active at once. In other words, these applications are more affected because they build large and changing working sets. Microsoft Word and Numbers are moderately affected. During ordinary editing, they usually remain stable and responsive. However, their memory demands increase when files become large, when many documents are open at once, when complex formatting is present, or when embedded media expands the working set. This means they are not as aggressive as Chrome or IDE-style environments in ordinary use, but they are still capable of showing clear slowdowns if memory pressure rises.

Lighter applications such as Finder, Calendar, Mail, TextEdit, and Terminal are less affected in normal use. Their working sets are smaller, their behavior is simpler, and they generally do not generate the same number of helper processes or large in-memory structures as browsers and development environments. This does not mean they are completely unaffected. If the whole system experiences severe memory exhaustion, even these lighter tools will slow down. The difference is comparative rather than absolute. They usually fail later, not never. The heaviest workload categories are even more sensitive than the desktop tools observed directly for this paper. Virtual machines, Docker or container stacks, video-editing software, large local AI models, and data-analysis jobs can allocate memory at a scale far beyond ordinary note-taking,

browsing, or document editing. Rupabhinda (2018) explains that virtualized workloads can become unstable or suffer performance penalties when memory pressure rises, and the system must reclaim or reallocate memory dynamically. Kanellopoulos et al. (2025) explain that virtual-memory overhead can consume a large fraction of CPU time when running large. This means that applications with large workloads are the ones most likely to reveal inadequate RAM capacity or heavy swap activity first. See Table 3 for the relative sensitivity of the main application types.

Table 3

Expected sensitivity to virtual-memory pressure by workload type

Sensitivity	Examples	Expected effect as pressure rises
High	Google Chrome, Code, Codex, virtual machines, Docker/container stacks, video editing, local AI models, large data-analysis jobs	More swap, long pauses, slower file indexing or rendering (image or videos), lower responsiveness time, application failures if RAM is exhausted
Moderate	Microsoft Word, Numbers, large PDFs, many simultaneous office documents	Slowdowns when switching documents, lag when editing, lower responsiveness
Lower in ordinary use	Finder, Calendar, Mail, TextEdit, Terminal	Responsiveness degrade if the system experiences sustained memory exhaustion

Note: The table summarizes the relative sensitivity of the main application types discussed in this paper.

When answering the question about which application types are affected and which are not by low RAM capacity and virtual memory, swapping. The best answer is that applications with large workloads and intensive memory requirements are affected first and affected more strongly, while applications with lighter workloads and memory requirements are less affected. This implies that virtual memory matters most when the workload is large and requires the

extensive use of system RAM. It matters less when the workload is small, simple, and mostly can reside within RAM.

Conclusion

As shown by the Apple documentation, supporting articles, and the examined macOS functionality, the Mac Studio machine is using a virtual memory configuration that is appropriate for its hardware and workload. The machine has an Apple M3 Ultra processor, 96 GB of RAM, secure virtual memory enabled, modest swap use, and a high free memory amount. This shows that virtual memory is active, but the system does not depend on it heavily during its ordinary usage and productivity work. The answer to whether the current setting of the machine virtual setting is optimal is yes; for this Mac, the current Apple-managed setting is effective and appropriate. The OS is using virtual memory in the way Apple intends, and the machine does not show that the system needs a larger manually controlled swap setting. In other words, the present configuration is more than sufficient for the observed workload. Additionally, The doubling and disabling virtual memory scenarios agree with this conclusion. Doubling virtual memory would mainly increase fallback capacity during unusual memory spikes, but it would not make the computer faster than RAM-based execution. Turning virtual memory off would be a riskier because it would remove a major stability and security mechanism implemented by the MacOS and a may result in memory-heavy workloads failing. This is especially important for applications such as Chrome (rendering videos and many tabs), Code, Codex app (AI agent), virtual machines, and other large-memory workloads, which are more sensitive to memory pressure than lighter tools such as Finder, Calendar, TextEdit, or Terminal. Ultimately, RAM size strongly may affects user experiences. Smaller-memory systems depend more heavily on swap (RAM/virtual memory swap), adding significant overhead that can affect user experience

negatively. Systems with large RAM capacities, such as the Mac Studio with 96 GB, still need virtual memory as a reserve and protection mechanism, rather than a constant memory-running mechanism required to maintain the system’s responsiveness.

References

Apple. (2003). *dynamic_pager(8)* [Manual page]. macOS.

Apple. (n.d.-a). *Check if your Mac needs more RAM in Activity Monitor*. Apple Support. https://support.apple.com/en-ie/guide/activity-monitor/actmntr34865/mac

Apple. (n.d.-b). *View memory usage in Activity Monitor on Mac*. Apple Support. https://support.apple.com/en-mide/guide/activity-monitor/actmntr1004/mac

Apple. (n.d.-c). *What is secure virtual memory on Mac?* Apple Support. https://support.apple.com/en-ie/guide/mac-help/mh11852/mac

Colorado State University Global. (n.d). *Lecture 5: Virtual memory* [Course lecture]. CSC507: Foundations of Operating Systems. Canvas.

Gupta, A. (2024, November). *Memory management technique in operating system* [Conference paper]. ResearchGate. https://www.researchgate.net/publication/385738258_Memory_Management_Technique_in_Operating_System

Kanellopoulos, K., Sgouras, K., Bostanci, F. N., Kakolyris, A. K., Konar, B. K., Bera, R., Sadrosadati, M., Kumar, R., Vijaykumar, N., & Mutlu, O. (2025). Virtuoso: Enabling fast and accurate virtual memory research via an imitation-based operating system simulation methodology. *In Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (pp. 1400-1421). https://doi.org/10.1145/3676641.3716027

Rupabhinda, S. (2018). A survey of memory management techniques for virtual machines. *International Journal of Novel Research and Development, 3*(5), 46-48. https://ijnrd.org/papers/IJNRD1805009.pdf